

1. C++ Language Overview

Course Information

- Author: Bjarne Stroustrup
- Created: Late 1980s (V2.0 1989, V2.1 1990, V3.0 1991)
- Instructor: J.M. Perronne
- Status: Standardized language (ISO 14882)
- Standards Published: C++98, C++03, C++11, C++14, C++17, C++21
- C++ is perceived as an evolution of C (M. Ritchie et B.W. Kernighan, early 1970s)
- Course covers compiled language concepts vs interpreted/scripting languages

Language Classification

- Modern C: Compiled language (not a scripting language)
- Applications: Games, finance, system programming
- Compilation Process: Source code → Object code (machine comprehension) → Executable code (linking, address relocation)

2. Program Structure and Modules

Anatomy of a C++ Program

- Main Program: Entry point of execution `main()`
- Modules: Separated into headers `(.h)` and implementation `(.cpp)`
- Module Components:
 - Header `(A.h)`: Interface contractual definition
 - Implementation `(A.cpp)`: Code object
 - Library `(L.h)`: Interface contractual of library, compiled code object

Compilation and Linking Process

1. Compilation: Header + Implementation → Code object
 2. Linking: Combines code objects into executable
 3. Preprocessor: Runs automatically before compilation
- Performs text transformations: file inclusion, text deletion, text replacement
 - Operations follow directives read from the file being analyzed

3. Preprocessor Directives

Definition The preprocessor is a program that analyzes a text file and performs transformations before compilation is executed.

Common Commands All commands begin with `#` at the start of a line

File Inclusion

```
#include "fileName"      // Normal file inclusion
#include <fileName>       // Standard library inclusion
```

Macro Definition and Text Replacement

```
#define aSymbol
#define First 1
```

Conditional Compilation

```
#ifdef identificateur
...
#endif
#ifndef (if not defined...)
#elif (else if...)
#if (if condition...)
```

Pragma Directive

```
#pragma arguments
```

- Provides compiler-specific instructions while maintaining code portability
- Not standardized across all compilers

Preprocessor Constants

- **LINE**: Current line number
- **FILE**: Current file name
- **DATE**: File processing date
- **TIME**: File processing time
- **__cplusplus**: Compiler identifier

Header Guard Pattern Prevents double inclusion of headers:

```
#ifndef TOTO_h
#define TOTO_h
// header content
#endif
```

Alternative (C++11): `#pragma once` (same effect but not in all compilers)

Assert Macro Example

- Used to verify conditions at runtime
- Assertion without code (if not defined)
- Standard direction on one line
- Can be removed dynamically without touching code

4. Data Types and Variables

Elementary Data Types

Type	Size (octets)	Range	Signification
char	1	-128 à 127	Character
unsigned char	1	0 à 255	Unsigned character
short	2	-32768 à 32767	Short integer
int	2/4	-2147483648 à 2147483647	Integer
unsigned int	4	0 à 4,294,967,295	Unsigned integer
long int	4	-2147483648 à 2147483647	Long integer
unsigned long int	4	0 à 4,294,967,295	Unsigned long integer
float	4	-3.410^{-38} à 3.410^{38}	Float (single precision)
double	4/8	-3.410^{-4932} à 3.410^{4932}	Double precision float
bool	varies	true/false	Boolean

Variable Declaration and Scope

- Scope: Between braces { ... } or global
- Points of declaration: No predefined locations
- Declaration within expressions: Allowed

```
int a;
int c, d;
for(int i=0; i < MAX; ++i) {...}
```

Variable Initialization

- With known values: `int a = 1;`
- With variable values: `int b = a;`
- Uniform initialization (C++11):
 - `int c{};` → initializes to 0
 - `int d{5};` → initializes to 5
- Note: Does not accept degrading conversions

Type Inference (C++11) auto Keyword

- Variable type determined by initializer
- Must be initialized at declaration
- Use only when necessary (can harm readability and maintainability)
- Justified for template usage

```
auto varInt = 12; // varInt is an integer
```

decltype Keyword

- Declares variable with identical type to another

```
decltype(varInt) anOtherVarInt = 25;
```

Creating New Types typedef

```
typedef type_original nouveau_type;  
typedef int Integer;
```

using (C++11)

```
using nouveau_type = type_original;  
using Integer = int;
```

Type Conversion Implicit Conversion

```
int i = 2.5; // Loss of precision
```

Explicit Conversion (Cast)

```
int i = (int) 2.5;
```

Constants Definition Methods

```
#define CTE 1 // Macro definition  
const int MAX = 10; // Constant variable  
const float CTEF = 1.0; // Float constant  
int tabInt[MAX]; // Valid in C++
```

const vs #define

- const provides type safety and proper scoping
- #define is text replacement by preprocessor
- Prefer const for type-safe constants

5. Operators

Operator Categories Arithmetic Operators +, -, *, /, %

Increment/Decrement Operators ++, -- (both prefix and postfix)

Comparison Operators ==, !=, >, <, >=, <=

Logical Operators || (OR), && (AND), ! (NOT)

Bitwise Operators &, |, ^, >>, <<

Assignment Operators +=, -=, *=, /=, %=, <<=, >>=, &=, ^=, |=

Ternary Operator (condition) ? result_if_true : result_if_false

Array Operator []

Cast Operator ()

Short-Circuit Evaluation

- Not all operands need evaluation
- (Exp Test) ? resultat_si_vrai : resultat_si_faux
- if (f || 28) && 8211 - if f is false, 8211 is not evaluated
- Useful for conditional expressions: int min = (a > b) ? b : a;

6. Control Structures

Modern for Loop (C++11) Based on interval iteration with range-based for:

```
int tabInt[] = {2,4,6,8,10};
for(int &v:tabInt)    std::cout << v << ' ';
```

- Using reference &v allows modification of elements
- Cleaner syntax than traditional for loops

7. Functions

Function Declaration and Definition Declaration (Header)

```
type_de_donnee Nom_De_La_Fonction(type1 argument1, type2 argument2, ...) {
    // Function body
}
```

Examples

```
void f();
float pi();
int max(int, int); // Function with two arguments
```

Function Calls

```
Nom_De_La_Fonction(); // No arguments
Nom_De_La_Fonction(argument1, argument2); // With arguments
int a = Nom_De_La_Fonction(); // Storing return value
```

Default Arguments

```
void printPrompt(char* p = "login > ") {
    cout << endl << p;
}
```

- Function can be called with or without arguments
- Defaults used when arguments not provided

Function Overloading C++ allows multiple functions with same name but different parameters:

```
void whoAmI(int i) { cout << "I am the integer :" << i << endl; }
void whoAmI(char c) { cout << "I am the char :" << c << endl; }
```

- Differentiation based on argument types/quantities
- Compiler selects appropriate version based on call

Function Pointers A pointer contains the address of the beginning of function binary code: Declaration

```
type (*identificateur)(type arg1, type arg2, ...);
int (*ptrFct) (int, int); // Pointer to function returning int with two
int params
```

typedef Usage

```
typedef int (*PtrFunc) (int, int);
```

Using Function Pointers

```
PtrFunc tabFuncs[]={subb, add, max};
for(int i=0; i<=sizeof(tabFuncs)/sizeof(PtrFunc); ++i)    cout << endl <<
do(tabFuncs[i],2,3);
```

8. Input/Output Operations

Output to Standard Stream cout and << Operator

```
#include <iostream>
using namespace std;
cout << "hello ";
int a = 1;
cout << a;
cout << endl << "a " << a;
```

Multiple Types

```
int n = 25;
char c = 'a';
double x = 3.15;
char *mess = "hello";
cout << "value of n : " << n << endl;
cout << "value of c : " << c << endl;
cout << "mess : " << mess << endl;
cout << "adresse de n : " << &n;
```

Input from Standard Stream cin and >> Operator

```
#include <iostream>
using namespace std;
int a;
cin >> a;
cout << endl << "a :" << a;
```

Reading Character Strings

```
char string[80];
for (unsigned int n=0; n<79; n++) {
    cin >> string[n];
    if (string[n]=='.') {
        string[n] = '\\0';
        break;
    }
}
```

```

    }
}
cout << "inputs : " << string;

```

9. References and Pointers

References Definition

- Alternative to pointers
- Represents an address (like a pointer)
- Manipulation acts on referenced object (not the reference itself)
- Must be initialized at declaration
- Can be perceived as constant pointer with dereferencing automatic
- Address cannot be changed after initialization
- Used as synonyms for variables

Reference Usage

```

int a = 1;
int &aref = a;
cout << aref << endl; // Output: 1
aref = 2;
cout << a << endl;    // Output: 2

```

Pointers Definition A pointer is a variable whose value is an address. Pointers are typed.

Declaration Examples

```

int *a;           // Pointer to integer
int b = 12;
a = &b;           // a takes address of b
cout << a;        // Display address: 0xbffffa7cc
cout << *a;       // Display content (value of b): 12

```

Pointer Arithmetic

```

int tab[5]={1,3,5,7,11};
int *ptr = tab; // Pointer takes value of tab
for(int i=0; i<5; ++i, ptr++) cout << ' ' << *ptr; // 1 3 5 7 11

```

Pointer Initialization (C++11)

- Before: `int *ptr_old = NULL;`
- Now: `int *ptr_new = nullptr; (type std::nullptr_t)`

Pointer Initialization (Modern)

- nullptr is preferred over NULL in C++11+

Arrays as Pointers "Un tableau est également un pointeur. L'arithmétique des pointeurs s'applique aux tableaux"

- Arrays are also pointers
- Pointer arithmetic applies to arrays

Passing Arguments to Functions By Pointer

```
void swap(int * a, int * b) {  
    int tmp = *a;  
    *a = *b;  
    *b = tmp;  
}  
int a=1, b=2;  
swap(&a, &b);  
cout << "a " << a << " b " << b; // Output: a 2 b 1
```

By Reference (Preferred)

```
void swap(int & a, int & b) {  
    int tmp = a;  
    a = b;  
    b = tmp;  
}  
int a=1, b=2;  
swap(a, b);  
cout << "a " << a << " b " << b; // Output: a 2 b 1
```

"On ne passe jamais en paramètre un objet par valeur !!"

- Never pass objects by value as parameters
- Can overflow the call stack
- Reference preferred over pointer
- In 9 out of 10 cases, pass by reference

10. Character Strings

String Definition in C++ A C++ character string is a character array or buffer terminated by null character '\0':

Pointer Declaration

```
char *mess = "hello";  
// Representation: 'h' 'e' 'l' 'l' 'o' '\0'
```

Array Declaration

```
char mess[] = {'h','e','l','l','o','\0'};
```

Special Characters

Character	Meaning
\0	Null character (string terminator)
"	Double quote
\	Backslash
\a	Alert/bell
\b	Backspace
\f	Form feed
\n	Newline
\t	Tab
\v	Vertical tab

String Library The library provides functions for character string manipulation:

- String copy
- String comparison
- String concatenation
- And more...

11. Complex Data Types

Arrays Declaration

```
type Nom_du_tableau [Nombre d'éléments]
int tabInt1[5]; // Array of 5 integers
int tabInt2[2] = {12, 5}; // Array initialized at declaration
int matrix[2][2] = {{12, 5},{3,8}}; // 2D array
```

Iteration

```
for(int i=0; i<2; i++)    cout << tabInt2[i];
// Output: 12 5
```

Structures Purpose Create a composite type where fields don't have the same type.

Declaration

```
struct NomStructure {  
    typeChamp1 champs1;  
    typeChamp2 champs2;  
    typeChampN champsN;  
};
```

Example

```
struct Point {  
    int x;  
    int y;  
};  
struct ColoredPoint {  
    Point point;  
    int color;  
};
```

Using Structures

```
Point p1;  
Point *p2 = &p1;  
p1.x = 10;  
p1.y = 20;  
ColoredPoint cp;  
cp.point.x = 10;  
cp.color = 0;
```

Operators

- Dot operator (.): Access members directly
- Arrow operator (->): Access members through pointer

```
Point *p2 = &p1;  
cout << p2->x // Equivalent to (*p2).x
```

Enumerated Types Definition Enumerated types are integral types (based on integers). Each enumeration value has a unique name.

Declaration

```
enum NomType {v1, v2, ..., vn};  
enum NomType {v1 = 5, v2, ..., vn};    // With initialization  
enum NomType {v1 = 5, v2 = 7, ...};    // Individual initialization
```

Example

```
enum Position {top, bottom, left, right};  
Position pos = left;  
switch (pos) {  
    case top: cout << "top"; break;  
    case bottom: cout << "bottom"; break;  
    case left: cout << "left"; break;  
    case right: cout << "right"; break;  
}
```

Strongly Typed Enums (C++11) Motivations

- No implicit conversions to integer
- Arithmetic impossible
- Ability to choose underlying integer size

Syntax

```
enum class Direction : short { up, down, left, right };  
Direction dir = Direction::up;  
switch (dir) {  
    case Direction::up : cout << "up"; break;  
    case Direction::down : cout << "down"; break;  
    case Direction::left : cout << "left"; break;  
    case Direction::right : cout << "right"; break;  
}
```

12. Dynamic Memory Management

Memory Allocation and Deallocation new Operator (Allocation)

```
new Type;           // Single object allocation  
new Type[n];        // Array allocation
```

delete Operator (Deallocation)

```
delete adresse;     // Single object deallocation  
delete[] adresse;   // Array deallocation
```

Example

```
const int N = 10;
int *ptr, *tab;
ptr = new int;
tab = new int[N];
delete ptr;
delete[] tab;
```

Placement new Two predefined placement operators that return the address passed as parameter (do not use delete):

```
new(adresse) type;
new(adresse) type[n];
```

13. Object-Oriented Programming

Classes and Objects Definition

- Class: Type, ensemble of objects with similar structure
- Object: Instance of a class, spatio-temporal entity

Class Components

- Identity: Differentiates object in class
- State: Attributes (data members)
- Behavior: Methods (member functions)

Class Declaration Structure Header File (.h)

```
#ifndef UN_SYMBOLE_UNIQUE
#define UN_SYMBOLE_UNIQUE
class NomClasse: public|protected|private NomClasseMere,... {
    public|protected|private:
        // Declaration of data members and/or methods
    public|protected|private:
        // Declaration of data members and/or methods
};
#endif
```

Implementation File (.cpp)

```
#include "nomfichier.h"
// Implementation of class methods
// Specific declarations
```

Access Modifiers Visibility Rules

Access Type	public	protected	private
public inheritance	public	protected	inaccessible
protected inheritance	protected	protected	inaccessible
private inheritance	private	private	inaccessible

Keywords

- public: Access for everyone
- protected: Access for same family classes
- private: Access restricted to the class itself

Encapsulation

- Support for encapsulation principle
- Access to entities defined by keywords
- Inheritance modulates visibility of ancestor components

Example: Point Class Header (point.h)

```
#ifndef POINT_H
#define POINT_H
class Point {
public:
    Point();
    Point(int, int);
    int getX() const;
    int getY() const;
    void setX(int);
    void setY(int);
private:
    int x, y;
};
#endif
```

Implementation (point.cpp)

```
#include "point.h"
Point::Point(): x(0), y(0) { }
Point::Point(int _x, int _y): x(_x), y(_y) { }
int Point::getX() const { return x; }
int Point::getY() const { return y; }
void Point::setX(int _x) { x = _x; }
void Point::setY(int _y) { y = _y; }
```

Usage (demo.cpp)

```
int main () {
    Point p1, p2(10,20);
    Point p3(p2);
    cout << "p1:" << p1.getX() << ',' << p1.getY();
    cout << "p2:" << p2.getX() << ',' << p2.getY();
    cout << "p3:" << p3.getX() << ',' << p3.getY();
    return 0;
}
// Output: p1:0,0  p2:10,20  p3:10,20
```

Constructors Three Types

1. Default Constructor: No parameters
2. Valued Constructor: With parameters
3. Copy Constructor: Takes reference to object of same class

Existence Rules

- Default constructor is generated as long as no constructor is explicitly defined
- Copy constructor is generated if not redefined
- Constructors invoke automatically during object instantiation

Behavior Rules

- Empty constructors (except copy) call parent class default constructor
- Initialize data members by invoking their default constructors
- Copy constructor calls parent class copy constructor
- By default, initializes data members "member to member"

Example with All Three

```
class Point {
public:
    Point();                // Default
    Point(int, int);        // Valued
    Point(const Point&);     // Copy
    ~Point();
};
Point::Point(): x(0), y(0) { cout << "-> défaut"; }
Point::Point(int _x, int _y): x(_x), y(_y) { cout << "-> valué"; }
Point::Point(const Point& p): x(p.x), y(p.y) { cout << "-> copie"; }
Point::~~Point() { cout << "-> destructeur"; }
```

Destructors Definition

- Function member called when object is destroyed
- Same name as class, preceded by ~

- No arguments, no return value
- Allows freeing of acquired resources

Existence and Behavior Rules

- Generated as long as not redefined
- Invoked automatically during object destruction
- Default behavior: invokes destructors of data members and parent class destructor

Usage with Dynamic Allocation

```
int main () {
    Point *p4 = new Point[3];
    Point *p5 = new Point(1,2);
    delete[] p4;
    delete p5;
    return 0;
}
```

Constant Members Constant Data Members Defined with const keyword:

```
class Truc {
public:
    Truc();
private:
    const int N;
};
Truc::Truc(): N(10) { }
```

- Initialized during construction
- Cannot be changed after initialization

Constant Member Functions Functions that must not modify object state:

```
class Point {
public:
    int getX() const;
    int getY() const;
};
```

- Used for consultation functions (Get methods)
- Example: getX() and getY() of Point class

Mutable Members Concept Constancy can be physical or logical:

- Object appears constant to clients
- May change internal state to ensure behavior

Example

```
class Point {
private:
    int x, y;
    mutable int access;
public:
    int getX() const {
        access++;
        return x;
    }
};
```

- Without mutable, compiler would forbid modification of access in const function
- Allows internal state changes in const functions

Static Members and Methods Definition

- Static members: Shared across all instances
- Static methods: Callable without instance creation
- Exist as soon as class exists (not dependent on objects)
- Each object possesses instances of members; static members are shared

Example: Object Counter

```
class Class {
public:
    Class();
    Class(const Class&);
    virtual ~Class();
    static unsigned int getRef();
private:
    static unsigned int ref;
};
unsigned int Class::ref = 0;
Class::Class() { ref++; }
Class::Class(const Class& c) { ref++; }
Class::~~Class() { ref--; }
unsigned int Class::getRef() { return ref; }
```

Usage with Inheritance

```
class Point: public Class { ... };
int main () {
    Point *p4 = new Point[3];
    // Creates 3 Point objects
    Point *p5 = new Point(1,2); // Creates 1 Point object
    Point p6(*p5);              // Copy constructor
```

```

    cout << Class::getRef();    // Output: 5
    delete[] p4;
    delete p5;
    cout << Class::getRef();    // Output: 1
    return 0;
}

```

Object Affection (Assignment) Default Behavior

- Default assignment operator exists
- Assignment is member-to-member
- Operator is redefinable

Redefinition Example

```

class Point: public Class {
public:
    Point& operator = (const Point&);
};
Point& Point::operator = (const Point& p) {
    if(&p != this) {
        x = p.x;
        y = p.y;
    }
    return *this;
}

```

Important Note "Les redéfinitions du constructeur par copie et de l'opérateur d'affectation vont de pair."

- Copy constructor and assignment operator redefinitions must go together
- Both should be redefined if managing dynamic memory

Type Conversion Implicit Conversion via Constructor

```

Point(int x, int y = 0); // Constructor definition
Point p = 2;             // Implicit conversion
cout << "p:" << p.getX() << ',' << p.getY(); // Output: p:2,0

```

Explicit Constructor

```

explicit Point(int x, int y = 0);
Point p = 2;           // Illegal
Point p = (Point) 2;   // OK

```

Conversion Operator

```
operator int ();
Point::operator int() {
    return sqrt(x*x + y*y);
}
Point p(2,2);
int i = p;
cout << i; // Output: 2
```

14. Inheritance and Polymorphism

Inheritance Basics Class Hierarchy Example

```
Class (Base)
├── Base
│   ├── Point
│   └── Rectangle
│       └── Square
```

Inheritance Definition

- Derived class: Inherits descriptions from parent class
- Multiple inheritance: Forbidden

Virtual Methods and Polymorphism Virtual Functions

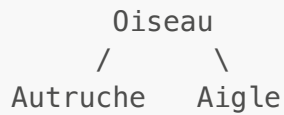
- Allow derived classes to override base class methods
- Virtual Pure: Abstract method (must be overridden)
- Polymorphism: Ability to have different behavior based on object type

Base Class Example

```
class Base : public Class {
public :
    virtual std::ostream& printOn(std::ostream&) const = 0;
    friend std::ostream& operator << (std::ostream&, const Base&);
};
std::ostream& operator << (std::ostream& os, const Base& o) {
    return o.printOn(os);
}
```

- Virtuelle pure: Abstract method (printOn)
- Classe Base is abstract (has pure virtual method)

Polymorphism Example: Birds



Behavior:

- Autruche: pas de vole (no flying)
- Aigle: vole (flies)

```

Oiseau * o1 = new Autruche();
Oiseau * o2 = new Aigle();
o1->vole(); // Depends if virtual
o2->vole();
  
```

Binding Types:

- Dynamic binding (if virtual): Calls derived class method
- Static binding (if not virtual): Calls base class method

Memory Management in Inheritance

```

delete o1; // Calls destructor (destructors not overridden typically)
  
```

- Use virtual in destructor to ensure proper cleanup
- Prevents memory leaks

Derived Class Implementation Point Class Inheriting from Base

```

class Point: public Base {
public:
    Point();
    Point(int, int);
    Point(const Point&);
    virtual ~Point();
    virtual Point& operator = (const Point&);
    virtual int getX() const;
    virtual int getY() const;
    virtual void setX(int);
    virtual void setY(int);
    virtual void translate(int, int);
    virtual std::ostream& printOn(std::ostream&) const;
private:
    int x, y;
};
  
```

Rectangle Class

```
class Rectangle: public Base {
public:
    Rectangle();
    Rectangle(const Point&, const Point&);
    Rectangle(int, int, int, int);
    virtual Point getTopLeft() const;
    virtual Point getBottomRight() const;
    virtual void setTopLeft(const Point&);
    virtual void setBottomRight(const Point&);
    virtual void translate(int dx, int dy);
protected:
    virtual std::ostream& printOn(std::ostream&) const;
private:
    Point topLeft, bottomRight;
};
```

Square Class with Private Inheritance

```
class Square: private Rectangle {
public:
    Square() {};
    Square(const Point&, int);
    Square(int, int, int);
    virtual Point getLocation() const;
    virtual int getSize() const;
    virtual void setLocation(const Point&);
    virtual void setSize(int);
    virtual void Translate(int, int);
    friend std::ostream& operator << (std::ostream&, const Square&);
};
```

Private Inheritance Note:

- Square "is-a" Rectangle internally
- Rectangle contract hidden and inaccessible
- Square contract completely redefined
- Public interface different from Rectangle

Default Constructor Delegation (C++11)

```
Point::Point(): Point(0,0) {}
```

- New feature allowing constructors to call other constructors
- Reduces code duplication

Exception Handling in Objects Relevant Cases

- Constructor: Exception during object creation
- Destructor: Should not throw (once instantiated)
- Input/Output Errors: Cannot be prevented
- Programming Errors: Can be corrected during development
- Assertions: If disabled, error handling code can be removed

15. Templates and Generic Programming

Function Templates Purpose Generate function code for various types (same function, different results):

- Type name as generic parameter
- Types generated based on function call input

Syntax

```
template<class T>
void swap(T& a, T& b) {
    T tmp = a;
    a = b;
    b = tmp;
}
template<class T>
T& max(T& a, T& b) {
    return (a > b) ? a : b;
}
template<class T>
void test(T& x, T& y) {
    cout << "x: " << x << " y: " << y;
    swap(x, y);
    cout << "x: " << x << " y: " << y;
    cout << "max(x,y): " << max(x, y);
}
```

Usage

```
int x = 1, y = 10;
test(x, y);
string s1 = "s1", s2 = "s2";
test(s1, s2);
// Output for integers:
// x: 1 y: 10
// x: 10 y: 1
// max(x,y): 10
```

Class Templates Dynamic Array Example

```

template <class T>
class Array: public Base {
public:
    typedef unsigned int size_a;
    Array();
    Array(size_a);
    virtual ~Array();
    virtual T& operator [] (size_a);
    virtual ostream& printOn(ostream &) const;
private:
    T *array;
    size_a size;
};

template <class T>
Array<T>::Array(): size(1), array(new T[1]) { }
template <class T>
Array<T>::Array(size_a sz): size(sz), array(new T[sz]) { }
template <class T>
Array<T>::~~Array() { delete[] array; }
template <class T>
T& Array<T>::operator [] (size_a i) {
    if (i > (size-1))
        throw "invalid index";
    return array[i];
}
template <class T>
ostream& Array<T>::printOn(ostream& s) const {
    for (size_a i=0; i<size; ++i)
        s << array[i] << ' ';
    return s;
}

```

Usage

```

try {
    Array<int> aint(3);
    aint[0] = 1;
    aint[1] = 2;
    aint[2] = 3;

    Array<string> astring(2);
    astring[0] = "st1";
    astring[1] = "st2";

    cout << aint << endl;
    cout << astring << endl;
    astring[3] = "st3"; // Throws "invalid index"
} catch(const char* msg) {
    cout << endl << msg;
}

```

Template Guidelines Important Notes

- Templates in headers: Templates must be compiled with each use
- typename vs class: Both can be used interchangeably
- Type requirements: T must support operations used in template (e.g., comparison)
- Comment on element: Document what type T needs to support

16. Exception Handling

Error Interception Mechanism Handles:

- Signal remontée toward calling functions
- Error signal detection by receptive function
- Appropriate treatment execution

Two-Phase Implementation Phase 1: Receptive Block

```
try {
    // Instructions susceptible to errors
}
```

Phase 2: Error Handlers

```
catch (TypeException1 e1) {
    // Instructions to handle exception
}
catch (TypeException2 e2) {
    // Instructions to handle exception
}
catch(...) {
    // Executed if no other catch matches
}
```

Error Triggering

```
throw exception_object;
```

Exception Class Hierarchy Base Exception

```
class exception {
public:
    exception() throw();
    exception(const exception &) throw();
    exception &operator=(const exception &) throw();
    virtual ~exception() throw();
}
```



```
virtual const char *what() const throw();
};
```

Exception Types Logic Errors (Programming errors):

- domain_error
- invalid_argument
- length_error
- out_of_range

Runtime Errors (Input/output):

- range_error
- overflow_error
- underflow_error

Custom Exception Example

```
class IllegalMonthException : public Base {
public:
    IllegalMonthException(unsigned char);
    virtual std::ostream& printOn(std::ostream&) const;
private:
    const unsigned short illegalMonth;
};
IllegalMonthException::IllegalMonthException(unsigned char m):
    illegalMonth(m) {}
std::ostream& IllegalMonthException::printOn(std::ostream& os) const {
    return os << "illegal month: " << illegalMonth;
}
```

Usage

```
const char* Monthconverter::convert(unsigned short m) {
    if(m < 1 || m > 12)
        throw IllegalMonthException(m);
    return month[m];
}
int main () {
    unsigned short month;
    while(true) {
        try {
            cout << "month to convert: ";
            cin >> month;
            cout << "-> " << Monthconverter::convert(month);
        } catch(const IllegalMonthException& e) {
            cout << endl << e;
        }
    }
}
```

```
    return 0;  
}
```

17. Standard Library

Categories Language Support

- Base Exceptions: `exception`, `bad_exception`, `bad_alloc`
- Complementary Exceptions: `domain_error`, `invalid_argument`, `logic_error`, `length_error`, `out_of_range`, `range_error`, `runtime_error`, `overflow_error`, `underflow_error`
- Dynamic Type Identification

General Utilities

- Operator Generators
- `pair` Structure
- Function Classes
- String Class

Containers Sequence Containers:

- `vector`: Continuous memory, immediate random access
- `list`: Sequential memory, no random access
- `deque`: Double-ended queue

Abstract Types:

- `queue`
- `priority_queue`
- `stack`

Associative Containers:

- `set`: Unique values
- `multiset`: Duplicate values allowed
- `map`: Key/value pairs (dictionary)
- `multimap`: Multiple values per key

Iterators

- Uniformly iterate over containers
- Same algorithms apply to all containers

Algorithms

- Components performing processing on/from containers

Numeric

- `complex`
- `valarray`

Input/Output

- ifstream: Input file stream
- istream: Input stream
- istreamstringstream: Input string stream
- ofstream: Output file stream
- ostream: Output stream
- ostreamstringstream: Output string stream
- ios_base: Base I/O stream class
- basic_ios: Basic I/O stream class

18. Input/Output Operations (Advanced)

Stream Format Manipulation File to Include

```
#include <iomanip>
```

Base Modifiers

- dec, hex, oct: Modify base for integer input/output
- showbase, noshowbase: Display/hide base indicator

Float Notation

- fixed, scientific: Fixed or scientific notation for floats
- showpoint, noshowpoint: Generate decimal point unconditionally

Alignment

- left, right: Left or right alignment

Sign Display

- showpos, noshowpos: Show/hide plus sign

Case

- uppercase, nouppercase: Uppercase/lowercase for hex letters

Whitespace

- skipws, noskipws: Ignore/respect whitespace in input

Width and Precision

- setw(int): Set field width
- setfill(int): Set fill character
- setprecision(int): Set decimal places

Flag Reset/Set

- `resetiosflags(ios_base::fmtflags)`: Reset specific flags
- `setiosflags(ios_base::fmtflags)`: Set specific flags

Examples

```
cout << "15 decimal-> " << 15;           // 15 decimal-> 15
cout << hex << "15 hexa-> " << 15;         // 15 hexa-> f
cout << showbase << hex << "15 hexa with base-> " << 15; // 15 hexa with
base-> 0xf
cout << fixed << "fixed notation-> " << 10.234; // fixed notation->
10.234000
cout << scientific << "scientific-> " << 10.234; // scientific->
1.023400e+001
cout << setfill('-') << setw(15) << 10.234; // -----10.234000
cout << setprecision(2) << "precision 2-> " << 10.234; // precision 2->
10.23
```

File Handling File to Include

```
#include <fstream>
```

Simple File Copy

```
ifstream fin("demo.txt");
ofstream fout("demo-copy.txt");
fin >> noskipws;
char c;
while (fin) {
    fin >> c;
    fout << c;
}
```

String Stream Operations File to Include

```
#include <sstream>
```

Value to String

```
template<class T>
string value2string(const T& v) {
    ostringstream os;
    os << v;
    return os.str();
}
```

String to Value

```
template <class T>
T string2value(const string& s) {
    istringstream is(s);
    T v;
    is >> v;
    return v;
}
```

Usage

```
string s("result: ");
cout << s + value2string(string2value<int>("2") + 3); // Output: result:
5
```

19. Lambda Expressions (C++11)

Anonymous Functions and Closures Purpose

- Functions without names (lambda expressions)
- Closures capture variables from outer scope
- Must specify captured variables

Basic Syntax

```
typedef std::function<int (int ,int)> PtrFunc;
PtrFunc max = [](int left ,int right)->int {
    return (left >= right) ? left : right;
};
PtrFunc add = [](int left ,int right)->int {
    return left + right;
};
std::cout << std::endl << max(12, 5); // Output: 12
```

Closures with Variable Capture

```
int x = 100, y = 200;
auto do_fermeture = [x, y](PtrFunc f)->int {
    return f(x, y);
};
PtrFunc tabFuncs[] = {max, add, subs};
for(PtrFunc &f : tabFuncs)
```

```
std::cout << std::endl << do_fermeture(f);
// Output: 200 300 -100
```

Capture Modes

- []: No external variables
- [x, &y]: x by value, y by reference
- [&]: All variables by reference
- [=]: All variables by value
- [&, x]: x by value, others by reference
- [=, &x]: x by reference, others by value

20. Containers and Iterators

Vector and List vector

- Memory: Continuous
- Access: Immediate (random access)
- Operations: Fast bulk addition/reading
- Speed: Optimized for sequential access

list

- Memory: Sequential
- Access: Random access not possible
- Operations: Fast point addition/reading
- Order: Maintains insertion order

Iteration Methods Traditional Iterator

```
vector<float> v1(tab, tab+5);
vector<float>::iterator it = v1.begin();
for(; it != v1.end(); ++it)    cout << *it << " ";
```

For-each with Lambda (C++11)

```
auto print = [](float f) { std::cout << f << ', '; };
std::vector<float> v1 = {1,2,3,4,5};
for(float v : v1)    std::cout << v << " ";
```

for_each Algorithm

```
void print(float f) { cout << f << ', '; }
for_each(v1.begin(), v1.end(), print);
// Or with lambda
for_each(v1.begin(), v1.end(), [](float f) { std::cout << f << ', '; });
```

Stream Iterators Output Stream Iterator

```
ostream_iterator<float> outPut(cout, "-");
copy(v1.begin(), v1.end(), outPut);
// Output: 1-2-3-4-5-
```

Input Stream Iterator

```
vector<float> v2;
istream_iterator<float> inIter(cin), endIter;
copy(inIter, endIter, back_inserter(v2));
```

Fixed-Size Arrays (C++11) Purpose Use arrays as STL containers

Syntax

```
#include <array>
std::array<float, 5> a1 = {1,2,3,4,5};
std::cout << "size : " << a1.size();
(a1.empty()) ? std::cout << "a1 empty " : std::cout << "a1 not empty ";
std::cout << "front() : " << a1.front();
std::cout << "back() : " << a1.back();
std::cout << "at() with limits: " << a1.at(1);
std::cout << "[] : " << a1[1];
copy(begin(a1), end(a1), std::ostream_iterator<float>(std::cout, "-"));
```

21. Advanced Techniques

Function Evaluation and Discrete Curves Evaluator Template

```
template <class T>
class Evaluator {
public:
    class EvalFunc {
    public:
        virtual T operator () (const T&) = 0;
    };

    typedef pair<vector<T>, vector<T>> Shape;

    static Shape buildShape(const T& min, const T& max, const T& step,
EvalFunc&);
    static ostream& printShape(ostream&, const Shape& s);
};
```

```
template <class T>
typename Evaluator<T>::Shape Evaluator<T>::buildShape(
    const T& min, const T& max, const T& step, EvalFunc& f) {
    vector<T> x, y;
    for (T i = min; i <= max; i += step) {
        y.push_back(f(i));
        x.push_back(i);
    }
    return Shape(x, y);
}
```

Triangle Function Example

```
template<class T>
class triangleFunc : public Evaluator<T>::EvalFunc {
public:
    triangleFunc(const T& _min, const T& _mid, const T& _max) :
        min(_min), mid(_mid), max(_max) {}

    virtual T operator() (const T& x) {
        if (x < min || x > max)
            return 0;
        return (x <= mid) ?
            (x - min) / (mid - min) :
            (max - x) / (max - mid);
    }
private:
    T min, mid, max;
};

void testShape() {
    triangleFunc<float> func(2.5f, 5, 7.5f);
    Evaluator<float>::Shape s = Evaluator<float>::buildShape(0, 10, 0.5f,
func);
    Evaluator<float>::printShape(cout, s);
}
```

Line Counting

```
#include <iostream>
#include <fstream>
#include <algorithm>
#include <iterator>
int main (int argc, char * const argv[]) {
    if (argc < 2) {
        cout << "filename missing";
        return 0;
    }

    ifstream file(argv[1]);
```



```

istream_iterator<char> it(file), itEnd;

if (file) {
    file >> noskipws;
    cout << "lines number: " << count(it, itEnd, '\n');
}
return 0;
}

```

Word Counter

```

class WordCounter {
public:
    WordCounter() = default;
    WordCounter(std::istream& s);
    void count(std::istream& s);
    std::ostream& printOn(std::ostream& os) const;
private:
    typedef std::map<std::string, unsigned long> w_map;
    typedef w_map::value_type w_value;
    typedef w_map::iterator iterator;
    typedef w_map::const_iterator const_iterator;
    w_map words;
};

WordCounter::WordCounter(std::istream& s) { count(s); }
void WordCounter::count(std::istream& s) {
    std::string current;
    while(s) {
        s >> current;
        (*words.insert(w_value(current, 0)).first).second++;
    }
}

std::ostream& WordCounter::printOn(std::ostream& os) const {
    for(const_iterator it = words.begin(); it != words.end(); ++it)
        os << (*it).first << "->" << (*it).second << std::endl;
    return os;
}

```

22. Practical Exercises and Assignments

TD HelloWorld First HelloWorld Simple program printing "Hello world" to standard output

Second HelloWorld Program using module for printing:

```

HelloWorld
├─ main() {...}
├─ helloworld.h (ostream& printHello(ostream&))
└─ helloworld.cpp (implementation)

```

TD DynamicArray Requirements Create DynamicArray class representing dynamic integer array:

- Instantiation: Default, with size specification, or by copy
- Information: Size and allocation block size
- Access: Get/set methods and operator[]
- Growth: Increase size on demand
- Output: Text representation on output stream

Generic Version Create Array template class:

- Same functionality as DynamicArray
- Works with any type T
- Supports dynamic sizing

TD Dictionary Class Diagram

```

Base
├── Dictionary
│   ├── - words: map<string, string>
│   ├── + Dictionary()
│   ├── + Dictionary(istream&)
│   ├── + Flush(): void
│   ├── + Size() const: unsigned int
│   ├── + Insert(const string&, const string&): void
│   ├── + Remove(const string&): void
│   ├── + Translate(const string&) const: string
│   ├── # ReadFrom(istream&): void
│   ├── # ReadPair(istream&): void
│   └── # PrintOn(ostream&) const: ostream&
└── Exception
    ├── - string: what
    ├── + Exception(const string&)
    ├── + What() const: string
    └── # PrintOn(ostream&) const: ostream&
        ├── DictionaryException
        │   └── + DictionaryException(const string&)
    
```

Dictionary Class Implementation

```

class Dictionary : public Base {
public:
    Dictionary();
    Dictionary(istream&);
    virtual ~Dictionary() {}
    virtual void flush();
    virtual unsigned int Size() const;
    virtual void insert(const string&, const string&);
    virtual void remove(const string&);
    
```

```
    virtual string translate(const string&) const;
protected:
    virtual ostream& printOn(ostream&) const;
    virtual istream& readFrom(istream&);
    virtual istream& readPair(istream&);
    friend istream& operator >> (istream&, Dictionary&);
private:
    typedef map<string, string> Words;
    typedef Words::value_type value_type;
    typedef Words::iterator iterator;
    typedef Words::const_iterator const_iterator;
    Words words;
};
```

23. C with C++ Integration

Using extern "C" Purpose Include C library headers in C++ code

Syntax

```
extern "C" {
    #include "alib.h"
    void f(); // C function
}
```

Notes

- Allows using C functions in C++ programs
- Different function name mangling between C and C++
- Must be explicitly declared to prevent linking errors